APPENDIX A

Proof of Theorem 1. When computing $N(e)$, we need to iterate through all vertices $u \in e$. Then, we use $E(u)$ to track every possible neighbor e' and calculate $OD(e, e')$. It takes $O(\delta d)$ time to process neighbors for one hyperedge. We need to invoke up to m times to traverse the whole hypergraph. Therefore, the total time complexity is $O(m\delta d)$. □

Proof of Lemma 1. This can be easily derived according to the concept of Definition 7. □

Proof of Lemma 2. ($\Rightarrow$) If e_w transitively covers $\{u, e, s\}$, then $e_w \overset{s}{\rightsquigarrow} e$ and $\mathcal{O}(e_w) < \mathcal{O}(e)$. Since $e_w \overset{s}{\rightsquigarrow} e$ and $e \overset{s}{\rightsquigarrow} e_u$, we have $e_w \overset{s}{\rightsquigarrow} e_u$. Therefore, e_w transitively covers $e \overset{s}{\rightsquigarrow} e_u$.

($\Leftarrow$) If e_w transitively covers $e \overset{s}{\rightsquigarrow} e_u$, then $e \overset{s}{\rightsquigarrow} e_w$, $e_w \overset{s}{\rightsquigarrow} e_u$ and $\mathcal{O}(e_w) < \mathcal{O}(e)$. Since $e_w \overset{s}{\rightsquigarrow} e_u$ and $e_u \in E(u)$, we have $u \overset{s}{\rightsquigarrow} e_w$. Therefore, e_w transitively covers $\{u, e, s\}$. □

Proof of Lemma 3. i) If e_u is of the highest importance in $W(e, e_u)$, that is, $\mathcal{O}(e_u) < \mathcal{O}(e)$, clearly, e_u transitively covers $\{u, e, s\}$.

ii) If e_w is of the highest importance in $W(e, e_u)$ and $e_w \neq e$, $e_w \neq e_u$, then $\mathcal{O}(e_w) < \mathcal{O}(e)$ and $\mathcal{O}(e_w) < \mathcal{O}(e_u)$. Since $WOD(W(e, e_u)) \geq s$, we have $e_u \overset{s}{\rightsquigarrow} e_w$ and $e_w \overset{s}{\rightsquigarrow} e$. Therefore, $u \overset{s}{\rightsquigarrow} e_w$, and e_w transitively covers $\{u, e, s\}$. □

Proof of Theorem 2. Suppose we have a pair of vertices u and v such that $MR(u, v) = s$ cannot be correctly answered based on $\mathcal{L}$. Since $MR(u, v) = s$, there should exist at least one walk $W(e_v, e_u)$ such that $u \in e_u$, $v \in e_v$ and $WOD(W(e_u, e_v)) = s$. Let e be the most important hyperedge across all such walks, we have $e \overset{s_u}{\rightsquigarrow} e_u$ and $e \overset{s_v}{\rightsquigarrow} e_v$ where $\min(s_u, s_v) = s$, and we have $(e, s_u) \notin \mathcal{L}(u)$, $(e, s_v) \notin \mathcal{L}(v)$ as $MR(u, v) = s$ cannot be correctly answered. Following Algorithm 2, we will not have (e, s_u) in $\mathcal{L}(u)$ only if either lines 8 or 14 holds. That is, either there exists a hyperedge e_w with higher importance than e such that $e_w \overset{s_u}{\rightsquigarrow} e$ and $e_w \overset{s_u}{\rightsquigarrow} e_u$, i.e., e_w is in a walk $W(e, e_u)$ with walk overlapping degree $WOD(W(e, e_u)) \geq s_u$; or there exists a hyperedge e_v with higher importance than e in the walk $W(e, e_u)$ with walk overlapping degree $WOD(W(e, e_u)) = s_u$. In both cases, e is not the most important hyperedge across all walks $W(e_v, e_u)$ with $WOD(W(e_u, e_v)) = s$, which contradicts our assumption. Therefore, the MR queries between any pair of vertices $u, v \in \mathcal{V}$ can be correctly answered by $\mathcal{L}(u)$ and $\mathcal{L}(v)$. □

Proof of Lemma 4. ($\Rightarrow$) If there exists a hyperedge e_w with $\mathcal{O}(e_w) < \mathcal{O}(e)$ that transitively covers $e \overset{s}{\rightsquigarrow} e_u$, then $e_w \overset{s}{\rightsquigarrow} e$, that is, there exists a walk $W(e_w, e)$ with $WOD(W(e_w, e)) = s$. Therefore, following the definition of maximum cover degree, $MCD(e) \geq s$.

($\Leftarrow$) Suppose $MCD(e) = k \geq s$, based on the definition of maximum cover degree, there exists a walk $W(e_w, e)$ with $\mathcal{O}(e_w) < \mathcal{O}(e)$ and $WOD(W(e_w, e)) = s$. Given $e \overset{s}{\rightsquigarrow} e_u$, i.e., there exists a walk $W(e, e_u)$ with $WOD(W(e, e_u)) = s$, then we can form a walk $W(e_w, e_u) = W(e_w, e) \oplus W(e, e_u)$ with $WOD(W(e_w, e_u)) = \min(k, s) = s$, that is, $e_w \overset{s}{\rightsquigarrow} e_u$. Together with $e_w \overset{s}{\rightsquigarrow} e$, we can conclude $e \overset{s}{\rightsquigarrow} e_u$ is transitively covered by e_w. □

Proof of Lemma 5. Based on Lemmas 2, 3 and the construction workflow mentioned in Algorithm 2, we early terminated the walk W that reaches hyperedge e when i) it only brings non-dominant tuples, and ii) it has been transitively covered by another hyperedge. Both cases indicate that we have explored another walk W' that reaches e with $WOD(W') \geq WOD(W')$. Suppose there exists a walk $W(e', e)$ that has been pruned in Algorithm 2, which affects the $MCD(e)$. According to Definition 8, this $W(e', e)$ has the highest walk overlapping degree among all walks in $\mathcal{W}(e_w, e)$ for all hyperedges e_w that satisfy $\mathcal{O}(e_w) < \mathcal{O}(e)$, which contradicts the pruning conditions in Algorithm 2, so the lemma holds. □

Proof of Lemma 6. Suppose $WOD(W(e', e_u)) = s_1$ and $OD(e_u, e_v) = s_2 \leq s$. Since $e \overset{k}{\rightsquigarrow} e_u$, there exists a walk $W(e, e_u)$ with $WOD(W(e, e_u)) = s$, and we can form two walks $W(e', e) = W(e', e_u) \oplus W(e_u, e)$ and $W(e, e_v) = W(e, e_u) \oplus \{e_v\}$. If $s_1 \geq s_2$, then we have $s_1 = s$, and $W(e, e') = \min(k, s_1)$ is s since $k \geq s$. Therefore, we have the lower bound of $MCD(e')$ is no smaller than s_2, so such $W(e', e_u) \oplus e_v$ will be early terminated at e_u. Another case is $s_1 < s_2$, in this case, the lower bound of $MCD(e')$ is s_1 derived from the walk $W(e, e_u) \oplus W(e_u, e')$, and such $W(e, e_u)$ with $WOD(W(e', e_u)) \leq MCD(e')$ will be early terminated. In both cases $e \overset{s}{\rightsquigarrow} e'$ and $e \overset{s}{\rightsquigarrow} e_v$, therefore $e' \overset{s}{\rightsquigarrow} e_v$ is transitively covered by e, so the lemma holds. □

Proof of Theorem 3. The time complexity of Algorithm 3 can be categorized into three parts, and we analyze them separately. i) Graph traversal: We start by analyzing the total number of elements pushed onto the priority queues. During the index construction from a hyperedge e, a label $(e, s) \in \mathcal{L}(u)$ is inserted (line 12) when u is first visited from e, that is, a hyperedge e_u that contains u is popped from the priority queue and being explored. Since each e_u is explored at most once from e, in the worst case, we explore all hyperedges in $E(u)$ and are only able to insert one label into $\mathcal{L}$. Thus, the total number of elements pushed onto and popped from the priority queues is bounded by $O(ld)$, and each push/pop operation takes $O(\log(ld))$ time given that the size of priority queues is at most $O(ld)$. Therefore, lines 7 and 21 take $O(ld \cdot \log(ld))$ time in total. Each time a hyperedge e_u is popped from the priority queues, line 10 iterates through all vertices in e_u and line 9 scans the neighbor-index of e_u, which takes $O(ld \cdot (\delta + \alpha_e))$ time in total. ii) Index update: Line 12 inserts a new label (e, s) into $\mathcal{L}(u)$ by appending it to the end of $\mathcal{L}(u)$, which takes constant time for each insertion and the overall index updating time for $\mathcal{L}$ is $O(l)$. iii) neighbor-index maintenance: The neighbor information for each hyperedge is computed exactly once (line 15), and the computation time is bounded by $O(m\delta d)$. The overall time complexity for deleting elements in neighbor-index is bounded by $O(\alpha \log \alpha_e)$. Therefore, the overall time complexity of Algorithm 3 is bounded by $O(ld \cdot (\log(ld) + \delta + \alpha_e) + m\delta d + \alpha \log \alpha_e)$. □

Proof of Theorem 4. According to Definition 5 and Lemma 3, we only add a label (e, s) into $\mathcal{L}(u)$ when e forms a walk $W(e, e_u)$ such that $u \in e_u$ and e has the highest importance among all hyperedges in $W(e, e_u)$. Note that, e_u can be e. Therefore, the number of labels in $\mathcal{L}(u)$ will be no greater than $|\mathcal{E}_{\leq u}|$ and the overall space complexity for our HL-index in both Algorithm 2 and 3 is bounded by $O(\sum_{u \in \mathcal{V}}(|\mathcal{E}_{\leq u}|))$. For a hyperedge e and the corresponding neighbor-index $\mathcal{M}(e)$, only those e_v with $OD(e, e_v) > MCD(e)$ will be maintained in $\mathcal{M}(e)$, so the number of neighbors of e maintained in $\mathcal{M}(e)$ will be a value α_e less than η_{max}. So the space complexity of the neighbor-index $\mathcal{M}$ is bounded by $O(m\alpha_e)$. □

Proof of Theorem 5. It is clear that the loops in line 4 and line 7 iterate l times since $\sum_{e \in \mathcal{E}} |\mathcal{D}(e)| \cdot |e| = l$, and the inner loop in line 5 iterates at most $O(l_v)$ times, Therefore, line 6 runs $O(l \cdot l_v)$ times in total to initialize $\mathcal{I}$. For each (u, s_u), the loop in line 9 iterates $O(l_v)$ times, the inner loop in line 10 iterates $O(\beta)$ times. The upper bound of β is derived from lines 4-6 of Algorithm 4, which is θ since the vertices in $\mathcal{I}(e)$ is the subset of vertices in $\mathcal{D}(e)$ based on Observation 1. Thus, lines 11-12 run $O(l \cdot l_v\beta)$ times in total, while each of them takes $O(\log \theta)$. So the overall time complexity of the loop in 9-13 is $O(l \cdot l_v\beta \cdot \log \theta)$. For lines 14-17, line 14 takes $O(\log \theta)$ time since it may search items in $\mathcal{D}(e)$. We can directly append (e, s_u) to the end of $\mathcal{L}^*(u)$ since we iterate over all hyperedges in descending importance order, as shown in line 15. line 16 takes $O(\theta \log \theta)$ time since we incur set interaction. Thus, lines 14-18 take $O(l \cdot (\log \theta + \theta \log \theta))$ time in total. Therefore, the overall time complexity of Algorithm 4 is bounded by $O(l \cdot (l_v\beta \log \theta + \theta \log \theta))$. □

Proof of Theorem 6. Evidently, Algorithm 4 will remove a label (e, s) from $\mathcal{L}(u)$ if and only if $MR(u, v)$ query for any $v \in \mathcal{V}$ can be correctly answered with other labels in the current $\mathcal{L}$.

First, we prove the completeness of $\mathcal{L}^*$ as follows. Suppose $MR(u, v) = k > 0$ cannot be correctly determined through $\mathcal{L}^*$, that is, there does not exist a hyperedge that supports $MR(u, v)$ in $\mathcal{L}^*$. According to Theorem 2, the input $\mathcal{L}$ is complete, therefore, there exists at least one hyperedge e that supports $MR(u, v)$ in the input $\mathcal{L}$ and we denote the set of such hyperedges as $\mathcal{E}_{u,v}$. $MR(u, v)$ cannot be derived from $\mathcal{L}^*$ only if the support for $MR(u, v)$ from each hyperedge $e \in \mathcal{E}_{u,v}$ is disrupted in Algorithm 4 when its essential tuples are being verified and $(e, *)$ is deleted either from $\mathcal{L}(u)$ or from $\mathcal{L}(v)$. Let the hyperedge e' be the last hyperedge in $\mathcal{E}_{u,v}$ to have its essential tuples verified. Then, the support from e' is disrupted if and only if $MR(u, v)$ is still supported by other hyperedges from $\mathcal{E}_{u,v}$ in the current $\mathcal{L}$, which contradicts our assumption. Therefore, $\mathcal{L}^*$ is complete.

Next, we prove $\mathcal{L}^*$ satisfies the minimality property. Suppose there exists a label $(e, s) \in \mathcal{L}^*(u)$ such that, for all $v \in \mathcal{V}$, $\mathcal{L}^*$ can still correctly answer $MR(u, v)$ after we (e, s) remove from $\mathcal{L}^*(u)$. Then, Algorithm 4 would have already removed (e, s) from $\mathcal{L}$, which contradicts our assumption. Therefore, $\mathcal{L}^*$ is minimal. □

Proof of Lemma 8. It can be easily derived according to the completeness property of our HL-index. □

Proof of Theorem 7. It is clear that to determine the existence of common hyperedge for both $\mathcal{L}(u)$ and $\mathcal{L}(v)$, we invoke a merge-sort-liked algorithm that need to iterate through elements in both $\mathcal{L}(u)$ and $\mathcal{L}(v)$ for exactly the once. Therefore, the time complexity of Algorithm 5 is $O(|\mathcal{L}(u)|+|\mathcal{L}(v)|)$. □

Proof of Obasevation 2. For every hyperedge $e = \{v\} \in \mathcal{E}$, e cannot transitively cover any other hyperedge since it has the lowest importance among all hyperedges in a walk that contains e. Besides, every walk W that contains e s.s. $W = \{e_1, e, e_2, ..., e_k\}$ can be simplified to $W' = \{e_1, e_2, ..., e_k\}$ while $WOD(W')$ remains unaffected, since we have $v \in e_2$. □

Proof of Observation 3. The proof for this observation is trivial and hence omitted.

□

APPENDIX B

Graph compacting. We notice that some hyperedges and vertices carry repeated or meaningless information, which could be a waste for both space and time efficiency. Therefore, we introduce the following steps to compact an input hypergraph, reducing the search space and accelerating the index constructing and querying stage. For a given hypergraph, we have the following two observations.

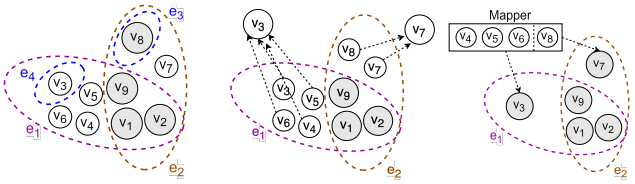
Observation 2. *For a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, all hyperedges with only one vertex inside carry no useful information.*

For a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, We assume that every $v \in \mathcal{V}$ must exist in at least one hyperedge, so the result of MR query for the same vertex will always be a non-zero value.

Observation 3. *For a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, if there are more than one vertex that exists and only exists in the same hyperedge, then those vertices carry the same information.*

Based on Observations 2 and 3, we propose the following graph compacting techniques. Given an input hypergraph, we first remove all hyperedges with the size of 1. Then for each hyperedge e, we scan through every vertex inside. All vertices inside e that do not exist in any other hyperedge are marked as isolated, which will then be mapped to the isolated vertex in e that has the smallest ID.

Example 10. Given the initial input hypergraph shown in Figure 5(a), we observe that there exist two hyperedges e_3, e_4 with the size of 1, so we first remove e_3 and e_4 from the hypergraph, and v_8 becomes an isolated vertex after that. We then use grey colour to represent all un-isolated vertices. According to the example of Figure 5(b). For each hyperedge, we map those isolated vertices into the one with the smallest ID. For example, all isolate vertices $\{v_3, v_4, v_5, v_6\} \in e_1$ are mapped to v_3, and $\{v_7, v_8\} \in e_2$ are mapped into v_7. In the final stage, all vertices are marked as un-isolated, and the hypergraph after compacting is shown in Figure 5(c).



(a) Original Graph (b) Compacting (c) Final Graph

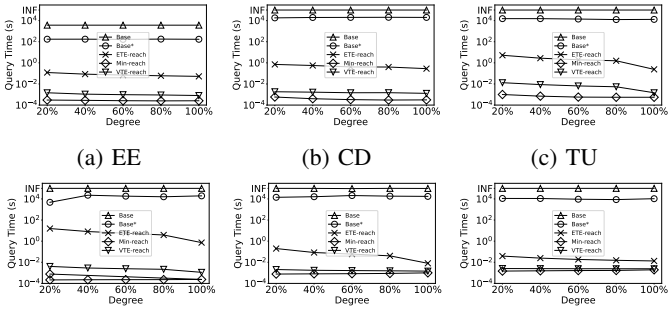
Fig. 5: Graph Compacting Procedure

APPENDIX C

We discuss and evaluate the VTV-based index, ETE-based index, and VTE-based index from the aspects of correctness, index construction time/space, and query performance. First of all, VTV-based index cannot guarantee the correctness of the max-reachability query, so we do not consider it as a valid solution. We will mainly compare the difference between ETE-based index and VTE-based index. From the space cost aspect, the cost of VTE-based index can be bounded by Algorithm 2, which is $O(\sum_{u\in\mathcal{V}}(|\mathcal{E}_{\leq u}|))$, where $\mathcal{E}_{\leq u} = \{e|e_u \in E(u) \wedge \mathcal{O}(e) \leq \mathcal{O}(e_u)\}$, as discussed in Theorem 4. The upper bound space cost of ETE-based index can be deduced in a similar way, where for every hyperedge e_u, it stores at most $|\mathcal{E}_{\leq e_u}|$, where $\mathcal{E}_{\leq e_u} = \{e|\mathcal{O}(e) \leq \mathcal{O}(e_u)\}$. The overall space cost for ETE-based index is then $\sum_{e\in\mathcal{E}}|\mathcal{E}_{\leq e_u}|$. Though we have $\mathcal{E}_{\leq e_u} \leq \mathcal{E}_{\leq u}$, in real world hypergraph datasets, $|\mathcal{E}|$ is much larger than $|\mathcal{V}|$. Therefore, the total space costs of VTE-based index and ETE-based index are close. For the construction time aspect, when constructing a complete index, the ETE-based index takes a factor of δ since we do not need to iterate through all vertices in e at line 9 of Algorithm 3. From the query efficiency aspects, suppose we are aiming to determine $MR(u, v)$. By adopting ETE-based index and applying the merge-sort-based algorithm. We should first iterate through all hyperedges in $E(u)$, shuffle and sort all elements inside $\mathcal{L}_u$, which is represented as $\mathcal{L}_u = \bigcup_{e_u\in E(u)}\mathcal{L}_e(e_u)$. Let $l_e^{max} = \max(|\mathcal{L}_e(e_u)|) : e_u \in \mathcal{E}$, building such $\mathcal{L}_u$ takes $O(d \cdot l_e^{max})$, where $d = \max_{v\in\mathcal{V}}|E(v)|$. Construct $\mathcal{L}_v$ for v that affords a similar time cost. Then the ETE-based $MR(u, v)$ query takes $O(|\mathcal{L}_u|+|\mathcal{L}_v|)$, and the overall time cost for ETE-based cost is bounded by $O(d\cdot l_e^{max}+|\mathcal{L}_u|+|\mathcal{L}_v|)$. Generally, $|\mathcal{L}_u|+|\mathcal{L}_v|$ is smaller than l_e^{max}, and for our VTE-based query, it takes $O(|\mathcal{L}(u)| + |\mathcal{L}(v)|)$, where we have $|\mathcal{L}(u)| << |\mathcal{L}_u|$ and $|\mathcal{L}(v)| << |\mathcal{L}_v|$. Therefore, our VTE-based approach outperforms ETE-based index for answering max-reachability queries in terms of query efficiency.

APPENDIX D

Exp-6: Query time by varying query vertices degree. We report the query time of all 5 query approaches on 6 datasets by varying the query vertices degree. Specifically, for each dataset, we first divide its vertices into 5 groups based on their degree, i.e., ($V_{20\%}$, $V_{40\%}$, $V_{60\%}$, $V_{80\%}$, $V_{100\%}$), where $V_{20\%}$ represents the top 20% of vertices with the highest degrees, and the same grouping method is applied to the other percentages. For each group, we randomly pick 1,000 queries inside and report the total processing time. Results are shown



(a) EE (b) CD (c) TU

(d) TS (e) TM (f) DB

Fig. 6: Total query time by varying query vertices degree

in Figure 6. As observed, Min-reach runs faster than all other algorithms by a significant margin, and the performance gap between Min-reach and ETE-reach increases as the degree of query vertices grows. For example, when queries from $V_{20\%}$ of dataset TM, Base*, ETE-reach, VTE-reach and Min-reach are 14,200s, 0.2s, 2ms and 0.6ms, respectively, while Base cannot terminate within 10 hours. Here, VTE-reach performs around 100x faster than ETE-reach, which is a more notable performance gap compared with queries from $V_{100\%}$, where VTE-reach takes 1.2ms and ETE-reach takes 9ms, respectively. We also observe that the response time of ETE-reach and VTE-reach decreases with the decrease in the degree of query vertices. For example, for the dataset TU, the total query time of on $V_{20\%}$ using ETE-reach and VTE-reach is 5.2s and 13ms, respectively. The value dropped to 0.24s and 1.5ms with queries from $V_{100\%}$. This is because for a vertex u, a higher degree implies a larger number of reachable hyperedges, resulting in more reachability tuples in a complete HL-index. We noticed that the query time of Min-reach is stable while the query vertex degree is varied, as entries corresponding to redundant vertex-to-hyperedge reachability information are removed to obtain a minimal HL-index.